

FINAL ASSIGNMENT FOR THE SEMESTER FINAL.

①

1. Demonstrate how a child class can access a protected member of its parent class within the same package. Explain with example what happens when the child class is in a different package.

Ans:

In Java, protected access modifier allows a member to be accessed:

- within the same package
- By subclasses, even if they are in different package

Case 1: Child class in the same package.

In this case, a protected member of a parent class is directly accessible in the child class without any special handling.

Example :

```
package pack1 // Parent.java
```

```
public class Parent {
```

```
    protected String message = "Hello from Parent!";
```

```
}
```

// File : Child.java

```
package pack1;
```

```
public class child extends Parent {
```

```
    public void displayMessage() {
```

```
        System.out.println(message);
```

```
}
```

```
public static void main (String[] args) {
```

```
    Child c = new child ();
```

```
    c.displayMessage();
```

```
}
```

```
}
```

Since child is in the same package, it can access the protected member ~~see~~ message directly.

Case 2: Child class in a different package.

Example:

```
package pack1;  
public class Parent {  
    protected String message = "Hello from Parent!"  
}  
  
Package Pack2;  
import pack1.Parent;  
public class Child extends Parent {  
    public void displayMessage() {  
        System.out.println(message);  
    }  
}
```

P.T.O

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

```
public static void main (String[] args) {  
    Child c = new Child ();  
    c.displayMessage();  
}
```

Child extends parent, so it can access the protected member via inheritance. But it cannot access the protected member using a parent class reference outside the package.

(2)

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q.2 Compare abstract classes and interfaces in terms of multiple inheritance. When would you prefer to use an abstract class and when an interface?

Answer: In Java, both abstract classes and interfaces provide abstraction, but they differ in design, usage and inheritance capabilities - especially in the context of multiple inheritance.

Feature	Abstract class	Interface
1.	Does not allow multiple inheritance	1. support multiple inheritance
2.	A class can extend only one abstract class.	2. A class can implement multiple interfaces se simultaneously

Example:

```
interface A {void showA();}
```

```
interface B {void showB();}
```

- class C implements A, B {

```
    public void showA() {System.out.println("A");}
```

```
    public void showB() {System.out.println("B");}
```

We

Use Abstract Class when,

- ① we want to share common code/logic
- ② we need constructors or protected methods
- ③ we are designing a base class
- ④ we want to maintain state or internal variables.

we use interface —

- (i) to define only method signature
- (ii) multiple inheritance
- (iv) to specify behavior contracts
- (v) to loose coupling and code flexibility.

3

Q.3 How ~~des~~ does encapsulation ensure data security and integrity? Show with a Bank Account class using private variables and validated methods such as set Account Number (String), set Initial Balance (double) that rejects null, negative or empty values.

Answer:

Encapsulation is a key principle of object-oriented programming that helps ensure data security and integrity by:

- i Hiding ~~inter~~ internal data using private variables.
- ii Controlling access to the data via public getter/ setter methods
- iii Adding validation, logic to prevent invalid data.

Example : (Bank - Account class)

```
public class BankAccount {  
    private String accountNumber;  
    private double balance;
```

```
    public void setAccountNumber (String accNum) {  
        if (accNum == null || accNum.trim().isEmpty())  
            System.out.println("Error")  
        } else {
```

```
            this.accountNumber = accNum;  
        }  
    }
```

TOPIC NAME : _____

DAY _____

TIME _____

DATE / /

```
public void setInitialBalance (double amount) {
```

```
    if (amount < 0) {
```

```
        System.out.println ("Error: Balance cannot  
        be negative.");
```

```
    } else {
```

```
        this.balance = amount;
```

```
    }
```

```
}
```

```
public String getAccountNumber() {
```

```
    return accountNumber;
```

```
}
```

```
public double getBalance() {
```

```
    return balance;
```

```
}
```

```
}
```

```
public class main {  
    public static void main (String [] args) {  
        BankAccount account = new BankAccount ();  
        account.setAccountNumber ("123456789");  
        account.setInitialBalance (1000.0);  
        System.out.println ("Account Number: " + account.  
            getAccountNumber());  
        System.out.println ("Balance: $" + account.getBalance());  
    }  
}
```

Q. 4: Write Program to - (Any 3)

① Find the kth smallest element in an ArrayList.

Ans:

```
import java.util.*;

public class KthSmallestElement {

    public static void main (String[] args) {
        ArrayList<Integer> list = new ArrayList<> (Arrays.
            asList (10, 4, 5, 8, 6, 11, 26));

        int k = 3;
        Collections.sort (list);
        System.out.println ("Sorted List : " + list);
        System.out.println (k + "rd smallest element: "
            + list.get (k-1));
    }
}
```

TOPIC NAME : _____

② Create a TreeMap to store the mappings of words to their frequencies in a given text.

Ans.

```
import java.util.*;

public class WordFrequency {
    public static void main (String[] args) {
        String text = "this is a test this is only a test";
        String[] words = text.split("\\s+");
        TreeMap<String, Integer> wordFreq =
            new TreeMap<>();
        for (String word : words) {
            wordFreq.put (word, wordFreq.getOrDefault
                (word, 0) + 1);
        }
        for (Map.Entry<String, Integer> entry :
            wordFreq.entrySet()) {
```

TOPIC NAME : _____

DAY _____

TIME _____

DATE / /

```
System.out.println(entry.getKey() + " -> " +  
entry.getValue());
```

```
}
```

```
}
```

④ Create a TreeMap to store the mappings of student IDs to their details.

Answer:

```
import java.util.*;
```

```
class Student {
```

```
    String name;
```

```
    int age;
```

```
    Student (String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

```
public String toString() {  
    return name + " (Age : " + age + " )";  
}
```

```
public class studentTreeMap {  
    public static void main (String[] args) {  
        TreeMap < Integer, student > studentMap =  
            new TreeMap<> ();  
  
        studentMap.put (102, new student ("Asha", 21));  
        studentMap.put (101, new student ("Maisha", 22));  
        studentMap.put (105, new student ("Nipa", 20));  
  
        for (Map.Entry < Integer, student > entry :  
            studentMap.entrySet()) {  
            System.out.println ("ID: " + entry.getKey() +  
                " -> " + entry.getValue());  
        }  
    }  
}
```

(5)

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q.5: Multithreaded car parking Management System using Synchronized Queue.

Answer:

1. RegistrarParking

2. ParkingPool

3. Parking Agent

4. Mainclass

1. RegistrarParking.java

```
public class RegistrarParking {  
    private final String carNumber;  
  
    public RegistrarParking (String carNumber) {  
        this.carNumber = carNumber;  
    }  
  
    public String getCarNumber() {  
        return carNumber;  
    }  
}
```

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

2. ParkingPool.java

```
import java.util.LinkedList;
import java.util.Queue

public class parkingpool {
    private final Queue<RegistrarParking> parkingQueue
= new LinkedList<>();

    public synchronized void requestParking (RegistrarParking
request) {
        parkingQueue.add(request);
        System.out.println ("Car" + request.getCarNumber() +
"requested parking.");
        notify();
    }

    public synchronized RegistrarParking getNextCarToPark()
{
        while (parkingQueue.isEmpty()) {

```

```
-try {
```

```
    wait();
```

```
} catch (InterruptedException e) {
```

```
    Thread.currentThread().interrupt();
```

```
} + a sleep + "long;" or using Thread.sleep();
```

```
}
```

```
return parkingQueue.poll();
```

```
}
```

3. ParkingAgent.java :

```
public class ParkingAgent extends Thread {
```

```
    private final int agentId;
```

```
    private final ParkingPool pool;
```

```
    public ParkingAgent (int agentId, ParkingPool pool)
```

```
    {
        this.agentId = agentId;
```

```
        this.pool = pool;
```

```
    }
```

@Override

```
public void run () {
```

```
    while (true) {
```

```
        RegisterParking request = pool.getNextCarToPark();
```

```
        System.out.println("Agent" + agentId + " parked car" +  
            + request.getCarNumber());
```

```
    try {
```

```
        Thread.sleep(1000);
```

```
    } catch (InterruptedException e) {
```

```
        Thread.currentThread().interrupt();
```

```
    }
```

```
}
```

```
}
```

```
}
```

4. MainClass.java :

```
public class MainClass {  
    public static void main (String[] args) {  
        ParkingPool pool = new ParkingPool ();  
  
        ParkingAgent agent1 = new ParkingAgent (1, pool);  
        ParkingAgent agent2 = new ParkingAgent (2, pool);  
        agent1.start();  
        agent2.start();  
  
        String[] carNumbers = {"ABC123", "XYZ456", "LMN789",  
                                "DEF321", "PQR654"};  
  
        for (String carNum : carNumbers) {  
            new Thread () -> {  
                RegistrationParking request = new RegistrationParking  
                    (carNum);  
                pool.requestParking (request);  
            }.start();  
        }  
    }  
}
```

TOPIC NAME : _____

DAY _____

TIME _____

DATE / /

try {

Thread.sleep(500);

} catch (InterruptedException e) {

Thread.currentThread().interrupt();

}

}

}

Q.6 How does Java handle XML data using DOM and SAX parsers? Compare both approaches with respect to memory usage, processing speed, and use cases. Provide a scenario where SAX would be preferred over DOM.

Answer:

Java provides two main ways to handle XML data:

1. DOM Parser
2. SAX Parser

1. DOM Parser in Java:

- Loads the entire XML document into memory as a tree structure.
- Provides easy navigation and manipulation of nodes.
- Part of the `javax.xml.parsers` package using `DocumentBuilder`.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : ____/____/____

2. SAX Parser in Java :

- Event-driven , serial access parser
- Doesn't load the entire file into memory
- Triggers events as it reads XML line by line .
- Part of org.xml.sax package .

Comparison Table : DOM vs SAX

Feature	DOM parser	SAX Parser
Memory Usage	High	Low
Speed	Slower for large files	Faster, especially with large XML
Access Type	Random access supported	Sequential access only
Node Modification	Supported	Not supported

When to prefer SAX over DOM : -

Scenario :

You're processing a 500MB XML file containing millions of transaction records and you only need to extract summary information.

Why SAX?

⇒ 1. DOM would require huge memory to load the full file.

2. SAX is more efficient for reading-only, stream-based processing.

Q.7 How does the virtual DOM in React improve performance? Compare it with the traditional DOM and explain the diffing algorithm with a simple component update example.

Answer:

The virtual DOM is a lightweight in-memory representation of the actual Real DOM used by React. It allows React to efficiently update only the parts of the UI that have changed instead of re-rendering the whole page.

How virtual DOM improves Performance?

1. React creates a virtual DOM tree from the component.
2. When the state or props change, a new virtual DOM is created.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : ____/____/____

3. React compares the new tree with the previous one.

4. Only the differences are applied to the real DOM

The process is called Reconciliation and avoids unnecessary DOM operations, making React faster

Traditional DOM vs Virtual DOM :

Feature	Traditional DOM	Virtual DOM
Update Mechanism	Direct manipulation of the actual DOM	works on virtual copy - first, then updates real DOM
Performance	Slower for frequent updates on large UIs	Faster due to minimized real DOM manipulation
Re-rendering	Entire components re-rendered	Only changed elements updated
Efficiency	Inefficient for complex UIs	Uses diffing and batching for efficiency

Q.8 What is event delegation in JavaScript, and how does it optimize performance? Explain with an example of a click event on dynamically added elements.

Answer:

Event delegation is a technique in JavaScript where a single event listener is added to a common parent of multiple child elements, instead of attaching event listeners to each child individually.

❖ Performance Benefits:

- Reduces memory usage by attaching fewer event listeners.
- Handles dynamically added elements without re-binding.
- Simplifies code, especially for long or changing

lists of elements.

Example : Click Event on Dynamically Added Buttons

HTML:

```
<div id = "button-container">  
</div>  
<button onclick = "addButton()">Add Button</button>
```

JavaScript:

```
const container = document.getElementById('button-  
container');  
container.addEventListener('click', function(event) {  
  if (event.target.tagName === 'Button') {  
    alert("You clicked: " + event.target.innerText);  
  }  
})  
  
function addButton() {  
  const newButton = document.createElement('button')  
  newButton.innerText = "Click Me" + Math.floor(  
    Math.random() * 100);  
  container.appendChild(newButton);  
}
```

Q.9 Explain how Java Regular Expressions can be used for input validation. Write a regex pattern to validate an email address and describe how it works using the pattern and Matcher classes.

Answer:

A regular Expression is a pattern that can be used to match character combinations in strings.

Java provides the `java.util.regex` package with two key classes.

- `Pattern` - compiles the regex
- `Matcher` - Applies the regex pattern to the input string.

A Regex used for validation to check the format of inputs like email, phone numbers, passwords etc. It reduces manual checking and complex conditional logic.

Email Validation Example:

Regex Pattern for Email:

```
String emailRegex = "^[a-zA-Z0-9.-%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$";
```

Pattern Explanation:

- (i) $\wedge$ → Start of String
- (ii) $[a-zA-Z0-9.-\%+-]^+$ → One or more valid characters for the local part
- (iii) $@$ → Mandatory @ symbol.
- (iv) $[a-zA-Z0-9.-]^+$ → Domain name

- ⑤ \. → Dot (.) between domain and extension
- ⑥ [a-zA-Z]{2,} → Extension like com, org etc.
- ⑦ \$ → End of string.

Java code Using Pattern and Matcher:

```
import java.util.regex.*;  
public class EmailValidator {  
    public static boolean isValidEmail (String email) {  
        String emailRegex = "^[a-zA-Z0-9.-%+]+@[a-zA-  
        -Z0-9.-]+\.[a-zA-Z]{2,}$";  
  
        Pattern pattern = Pattern.compile(emailRegex);  
        Matcher matcher = pattern.matcher(email);  
        return matcher.matches();  
    }  
  
    public static void main (String[] args) {  
        String email1 = "asha25@example.com";  
        String email2 = "invalid-email@";
```

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

```
System.out.println(email1 + "is valid:" + isValidEmail  
(email1));
```

```
System.out.println(email2 + "is valid:" + isValidEmail  
(email2));
```

```
}
```

```
}
```

Java's Pattern and Matcher classes offer a powerful way to validate inputs using regex. In this example, we validated email addresses efficiently by applying a regex pattern to check proper format and structure.

Q.10 What are custom annotations in Java, and how can they be used to influence program behaviour at runtime using reflection? Design a simple custom annotation and show how it can be processed with annotated elements.

Ans:

Custom annotations in Java are user defined metadata that can be attached to classes, methods, fields or other elements. They are created using the `@interface` keyword and can be used to

- Add metadata to code elements
- Influence program behaviour during runtime reflection.

How annotations work with Reflection:

Java's `java.lang.reflect` package allows programs to:

- Inspect annotations at runtime
- React based on the annotation's values.

1. Define a custom Annotation:

```
import java.lang.annotation.*;  
@Retention(RetentionPolicy.RUNTIME)  
@Target(ElementType.METHOD)  
public @interface RunThis {  
    int times() default 1;  
}
```

2. Use the Annotation in a class :

```
public class MyTasks {  
    @RunThis(times = 3)  
    public void greet() {  
        System.out.println("Hello from greet()");  
    }  
  
    @RunThis  
    public void sayBye() {  
        System.out.println("Goodbye from SayBye()");  
    }  
}
```

```
public void ignoreThis() {  
    System.out.println("This should not run.");  
}  
}
```

3. Process the Annotation Using Reflection :

```
import java.lang.reflect.Method;  
public class AnnotationProcessor {  
    public static void main (String[] args) throws Exception {  
        MyTasks obj = new MyTasks();  
        for (Method method : obj.getClass().getDeclaredMethods()) {  
            if (method.isAnnotationPresent(RunThis.class)) {  
                RunThis annotation = method.getAnnotation(RunThis.class);  
                int times = annotation.times();  
                for (int i = 0; i < times; i++) {  
                    method.invoke(obj);  
                }  
            }  
        }  
    }  
}
```

Expected Output:

Hello from greet()

Hello from greet()

Hello from greet()

Goodbye from sayBye().

(11)

Q.11 . Discuss the Singleton design pattern in Java. What problem does it solve, and how does it ensure only one instance of a class is created? Extend your answer to explain how thread safety can be achieved in a Singleton implementation.

Ans:

The Singleton Design Pattern ensures that only one instance of a class is created throughout the application and provides a global point of access to it.

Problem it solves:

P.T.O

- prevents multiple instance of a class where only one shared object is needed
- Controls resource usage and ensures consistent behavior.

Basic Singleton Implementation :

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Thread Safe Singleton :

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Efficient Thread Safe Singleton :

```
public class Singleton {  
    private static volatile Singleton instance;  
    private Singleton() {}  
}
```

```
public static Singleton getInstance() {  
    if (instance == null) {  
        synchronized (Singleton.class) {  
            if (instance == null) {  
                instance = new Singleton();  
            }  
        }  
    }  
    return instance;  
}
```

Q.12 Describe how JDBC manages communication between a java application and a relational database. Outline the steps involved in executing a ~~set~~ SELECT query and fetching results. Include error handling with try-catch and finally blocks.

Ans:

JDBC is an API that enables java applications to communicate with relational databases.

How JDBC works:

JDBC provides a standard interface to connect, send queries, and process results from a database using.

- Driver Manager
- Connection
- Statement
- ResultSet

- Steps to execute a SELECT Query using JDBC:

```
import java.sql.*;
```

```
public class FetchDataExample {
```

```
    public static void main (String[] args) {
```

```
        Connection conn = null;
```

```
        Statement stmt = null;
```

```
        ResultSet rs = null;
```

```
        try {
```

```
            Class.forName ("com.mysql.cj.jdbc.Driver");
```

```
            conn = DriverManager.getConnection (
```

```
                stmt = conn.createStatement ();
```

```
                String query = "Select id, name FROM students";
```

```
                rs = stmt.executeQuery (query);
```

```
                while (rs.next()) {
```

```
                    int id = rs.getInt ("id");
```

```
                    String name = rs.getString ("name");
```

```
System.out.println("ID:" + tid + "Name:" + name);  
}  
{ catch (SQLException e) {  
    System.out.println("SQL Error:" + e.getMessage());  
}  
{ catch (ClassNotFoundException e) {  
    System.out.println("Driver not found!");  
}  
finally {  
    try {  
        if (rs != null) rs.close();  
        if (stmt != null) stmt.close();  
        if (conn != null) conn.close();  
    } catch (SQLException e) {  
        System.out.println("Error closing resources:" +  
            e.getMessage());  
    }  
}  
}
```

Q. 13. How do Servlets and JSPs work together in a web application following the MVC architecture? Provide a brief use case showing the servlet as a controller, JSP as a view, and a Java class as the model.

Answer:

In the MVC architecture:

- Model : Handles business logic and data
- View : Displays data to users.
- Controller : Handles user requests and controls the flow.

How They work Together:

1. User sends a request to a servlet
2. The Servlet (controller):
 - Reads input parameters
 - Calls the Model to fetch or process data.

- Stores data in request scope.
- Forwards the request to a JSP for rendering.

3. Jsp accesses the data and displays it to the user

* Use Case Example : Show User Profile by ID.

Model - User.java

```
public class User {  
    private int id;  
    private String name;  
    public User(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
    public int getId() { return id; }  
    public String getName() { return name; }  
}
```

controller - UserServlet - Java :

@WebServlet ("user")

public class UserServlet extends HttpServlet {
protected void doGet (HttpServletRequest request,
HttpServletResponse response)

throws ServletException, IOException {

int userId = Integer.parseInt (request.getParameter("id"));

User user = new User (userId, "Asha");

request.setAttribute ("user", user);

RequestDispatcher rd = request.getRequestDispatcher

("userView.jsp");
rd.forward (request, response);

}

}

View - userView.jsp :

```
<%@ page contentType = "text/html" %>
<%@ page import = "yourpackage.user" %>
<%
    User user = (User) request.getAttribute("user");
%>
<html>
<head> <title> User Profile </title> </head>
<body>
    <h2> User Details </h2>
    <p> ID: <%= user.getId() %> </p>
    <p> Name: <%= user.getName() %> </p>
</body>
</html>
```

Q.14 Explain the life cycle of a java servlet.
What are the roles of the `init()`, `service()`, and `destroy()` methods? Discuss how servlets handle concurrent requests and how thread safety issues arise.

Answer:

A java Servlet follows a specific life cycle managed by the Servlet container. The life cycle includes loading, initialization, request handling and destruction.

Servlet Life cycle stages:

1. Installation: Servlet container creates a single instance of the servlet.
2. Initialization & `init()`: Called once when the servlet is first loaded.

3. Request Handling (service) : called for each request.

4. Destruction (destroy()) : called once before servlet is removed from memory.

1. init (ServletConfig config)

public void init (ServletConfig config) throws
ServletException {

}

- called for each client request
- Dispatches the request to doGet(), doPost(), etc.
- Handles the core logic of the application.

2. service (HttpServletRequest req, HttpServletResponse res)

- Called for each client request
- Dispatches the request to doGet(), doPost() etc
- Handles the core logic of the application.

```
protected void service(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException {
}
```

3. destroy()

- Called once before servlet is unloaded
- Used to release resources.

```
public void destroy() {
}
```

Handling concurrent Requests :

- By default, servlets are multithreaded
- A single servlet instance handles multiple threads simultaneously
- The `service()` method runs in multiple threads concurrently.

Thread Safety Issues:

- Shared mutable fields can lead to race conditions and data inconsistency

- Example of non-thread-safe code:

```
private int counter = 0;

protected void doGet (HttpServletRequest request,
    HttpServletResponse response) {
    counter++;
}
```

To ensure Thread Safety:

- Avoid using shared mutable state
- Use local variable inside methods
- Use synchronization or concurrent utilities if shared state is required.

15

Q.15

A single instance of a servlet handles multiple requests using threads. What problems can occur if shared resources are accessed by multiple threads? Illustrate your answer with an example and suggest a solution using synchronization.

Answer:

In a Java servlet, only one instance of the servlet class is created by the container and it handles multiple requests concurrently using multiple threads. If the servlet uses shared variables, then multiple threads can access or modify these resources at the same time, leading to race conditions, inconsistent data and unexpected behavior.

Problem: Race condition with shared variable

Example: Non-thread safe counter.

```
public class CounterServlet extends HttpServlet {  
    private int counter = 0;  
  
    protected void doGet (HttpServletRequest request,  
        HttpServletResponse response) throws ServletException,  
        IOException {  
    }
```

```
    counter++;  
    response.getWriter().println("Visitor count: " + counter);  
}  
}
```

- Two threads might read and update the value of counter at the same time.
- This may cause some increments to be lost, resulting in an incorrect count.

Synchronization:

To ensure that only one thread at a time accesses the critical section, use 'synchronized' blocks or methods.

Synchronized Version :

```
public class CounterServlet extends HttpServlet {  
    private int counter = 0;  
    protected void doGet(HttpServletRequest request,  
        HttpServletResponse response)  
        throws ServletException, IOException {  
        synchronized (this) {  
            counter++;  
        }  
        response.getWriter().println("Visitor count: " +  
            counter);  
    }  
}
```

The `synchronized (this)` block locks the servlet object so only one thread can increment the counter at a time.

Q.16

Describe how the MVC pattern separates concerns in a Java web application. Explain the advantages of this structure in terms of maintainability and scalability, using a student registration system as an example.

Answer:

The MVC pattern is a design approach that separates a web application into three interconnected components.

Separation in concerns :

1. Model → Manages the data, business logic and rules of the application.

2. View - Represents the presentation Layer (HTML/JSP pages shown to the user)
3. Controller - Handles user input, manages request and determines what response to send.

How MVC works:

When a user interacts with the application:

1. Controller receives the request
2. It interacts with the Model to process data
3. It forwards the result to a View.

Example: Student Registration System.

Use Case: A student submits their name and email via a form.

Model: Student.java

```
public class student {  
    private String name;  
    private String email;  
    public student (String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

View : register.jsp

```
<form action = "RegisterStudentServlet" method = "post">  
    Name : <input type = "text" name = "name"> <br>  
    Email : <input type = "text" name = "email"> <br>  
    <input type = "submit" value = "Register">  
</form>
```

Controller : RegisterStudentServlet.java

@webServlet (" / RegisterStudentServlet")

public class RegisterStudentServlet extends HttpServlet {
protected void doPost (HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {

String name = request.getParameter ("name");

String email = request.getParameter ("email");

Student student = new Student (name, email);

request.setAttribute ("student", student);

RequestDispatcher dispatcher = request.getRequestDispatcher ("success.jsp");

dispatcher.forward (request, response);

}

}

success.jsp:

<p>student Registered Successfully! </P>
<p>Name : \$ {student.name} </P>
<p>Email : \$ {student.email} </P>

2. Advantages of MVC Architecture:

Benefit	Description
1. Maintainability	• to change the view without affecting the model or controller.
2. Scalability	• Easy to add new features by extending each layer separately.
3. Testability	• Business logic in the model can be tested independently from UI and controllers.
4. Reusability	• Models and views can be reused across multiple controllers or modules.

In a student registration system MVC ensures that,

- Data is isolated and reusable
- UI is clean and only handles display logic
- Controller handles flow and decision-making, linking Model and View.

(17)

Q.17:

In a Java EE application, how does a servlet controller manage the flow between the model and the view? Provide a brief example that demonstrates forwarding data from a servlet to a JSP and rendering a response.

Solution:

P.T.O

In a Java EE application, a servlet controller plays a central role in managing the flow between the Model and the View using the MVC pattern.

Role of Servlet Controller in MVC:

1. Receives user input
2. Interacts with the Model
3. Stores data in request scope
4. Forwards the request to the view (JSP) using a Request Dispatcher.

Example : Servlet Controller Forwarding Data to JSP .

1. Model :

```
public class User {  
    private String name;  
  
    public class User (String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

2. View :

```
<%@ page contentType = "text/html" ; charset = UTF-8"%>  
<html>  
  <head><title>Welcome </title> </head>  
  <body>  
    <h2>Welcome , $ { User.name } ! </h2>  
  </body>  
</html>
```

3. Servlet Controller (WelcomeServlet.java)

@WebServlet("WelcomeServlet")

public class WelcomeServlet extends HttpServlet {
protected void doPost (HttpServletRequest request,HttpServletResponse response) throws ServletException,
IOException {

String name = request.getParameter("name");

User user = new User(name);

request.setAttribute("User", user);

RequestDispatcher dispatcher = request.getRequestDispatcher("welcome.jsp");

dispatcher.forward(request, response);

}

}

Form Page (index.jsp or form.html):

```
<form action = "WelcomeServlet" method = "post">  
  Enter your name : <input type = "text" name  
    = "name" >  
  <input type = "submit" value = "submit">  
  
</form>
```

The Servlet acts as the controller, receives the user input, creates a Model object (User), and stores it in the request.

It then forwards the requests to the View `welcom.jsp` where the data is rendered using JSP EL (`${user.name}`).

Q.18

Compare and contrast cookies, URL rewriting and HttpSession as methods for session tracking in servlets. Discuss their advantages, limitations, and ideal use cases.

Solution:

A cookie is a small ~~pie~~ piece of data stored on the client's browser and sent with every request to the server.

Advantages:

1. Easy to use and supported by all browsers
2. Can persist beyond the session.

Limitations:

1. Can be disabled by users
2. Visible to users → potential security risks
3. Size limit
4. Slower if too many cookies are present.

Ideal-Use-cases:

1. Storing non-sensitive data
2. Remembering users between visits.

Example:

```
Cookie cookie = new Cookie("username", "anha");  
response.addCookie(cookie);
```

2. URL Rewriting:

Appends session data to the URL as a query string.

Example:

```
http:// example.com/page.jsp?jsessionid = 1234ABCD
```

Advantages:

- Works even if cookies are disabled
- Simple to implement if all links are under control

Limitations:

- Session info is visible in the URL → less secure
- Doesn't work for external links
- Extra effort to rewrite every URL manually or using helper methods.

Ideal Use Cases:

- Applications where cookies are disabled or
- Lightweight apps with limited navigation and few external links.

Example:

```
String encodedURL = response.encodeURL("/dash  
dashboard.jsp")  
out.println("<a href = " + encodedURL + ">Go  
to Dashboard </a>");
```

3. HTTP Session :

A server-side session tracking mechanism. The server creates a HttpSession object for each user.

Advantages :

- Most secure method
- Allows storing complex java objects
- Automatic session ID handling with cookies or URL rewriting.

Limitations :

- Consumes server memory
- Requires session cleanup

Ideal Use Cases :

- E-commerce apps
- Applications that require secure, long-lived user sessions.

Example:

```
HttpSession session = request.getSession();  
session.setAttribute("user", "nasha");
```

(19)

Q.19

A web application stores user login information using HttpSession. Explain how the session works across multiple requests and how session timeout or invalidation is handled securely.

Solution:

How HttpSession works Across Requests:

1. Session Creation:

When a user logs in or accesses the app for the first time, a session is created.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

```
HttpSession session = request.getSession();  
session.setAttribute("username", "asha");
```

2. Session Identification:

- The server generates a unique session ID.
- On each subsequent request, the browser sends this session ID back to the server.
- The server uses this ID to retrieve the correct session object.

3. Session Persistence:

As long as the session ID is valid and the session hasn't timed out or been invalidated, the server retains access to session's attributes.

Session Timeout and Invalidation :

1 Automatic Timeout

- By default, sessions expire after 30 minutes of inactivity
- This value can be configured.

In web.xml:

```
<session-config>  
  <session-timeout>15</session-timeout>  
</session-config>
```

2. Manual Invalidation:

To log a user out explicitly :

```
session.invalidate();
```

~~Exam~~

Secure Handling of Sessions:

1. Use HTTPS
2. Regenerate session ID after Login.
3. Limit session duration
4. Avoid storing sensitive info
5. Logout handling

Explain how spring mvc handles an HTTP request from a browser. Describe the role of the @controller, @RequestMapping and Model objects in separating business logic from presentation. Provide a brief flow & example of a login form submission.

Solution:

Spring MVC is a web framework based on the Model-View-Controller design pattern. It cleanly separates responsibilities:

Model : Data or business logic

View : Presentation layer

Controller : Handles user input and updates the model and view accordingly.

How Spring MVC Handles an HTTP Request: -

1. Client sends Request : A browser sends an HTTP request to a spring MVC application.
2. Dispatcher Servlet intercepts Request : The Dispatcher Servlet is the front controller. It intercepts the request and routes it.
3. @Controller Handles the Request : Based on the URL pattern, the DispatcherServlet finds a matching method in a class annotated with @controller.
4. @RequestMapping Maps the URL : The method inside the controller is annotated with @RequestMapping to handle a specific URL and HTTP method.

5. Model is Populated:

The controller method uses a Model object to pass data to the view success/failure message, user info.

Example : Login form submission flow -

Step 1: HTML Form (login.jsp)

```
<form action = "login" method = "post" >  
  <input type = "text" name = "username" / >  
  <input type = "password" name = "password" / >  
  <button type = "submit" > login </button>  
</form>
```

Step 2: Controller

```
@Controller  
public class LoginController {
```

@PostMapping ~~url~~ ("/login")

public String handleLogin (

@PostMapping ("/login")

public String handleLogin (

@RequestParam String username,

@RequestParam String password,

Model model) {

if ("admin".equals(username) && "1234".equals(password)) {
 model.addAttribute("username", username);
 return "welcome";

} else {

 model.addAttribute("error", "Invalid credentials");
 return "login";

}

}

}

Step 3: welcome.jsp

`<h2>Welcome, $ {username}! </h2>`

Benefits of Spring MVC:

Separation of concerns: Clean division of logic, data and view.

Testable: Each component can be tested separately

Scalable: New features can be added without breaking existing code.

Maintainable: Easier to debug and maintain due to modular design.

Q.29 Spring MVC uses the DispatcherServlet as a front controller. Describe its role in the request processing workflow. How does it interact with view resolvers and handler mapping?

Solution:

The DispatcherServlet coordinates the entire workflow from receiving a request to rendering a response. Here's a step by step breakdown:

Request Processing Workflow:

1. Client Request
2. DispatcherServlet Receives Request
3. Handler Mapping Lookup
4. Controller Execution.
5. Model and View Generation
6. View Resolver
7. Render Response

Example: Spring MVC with JSP

Suppose a login request hits /login.

1. URL HH:

POST/login

2. Dispatcher Servlet: Mapped to handle all requests.

3. HandlerMapping:

```
@PostMapping("/login")
```

```
public String login(@RequestParam String  
username, Model model)
```

4. Controller Returns:

```
return "welcome";
```

5. ViewResolver Translates:

- Logical name: "welcome"
- To physical view: /WEB-INF/views/welcome.jsp.

6. Render: Data from model injected into welcome.jsp, HTML sent back.

(22)

Q.22 How does Prepared Statement improve performance and security over statement in JDBC? write a short example to insert a record into a MySQL table using Prepared Statement.

Solution:

Prepared Statement improves security and performance over Statement by:

- Preventing SQL injection through parameterized queries
- Reusing compiled SQL, which is faster.
- Making code cleaner and type-safe.

Example (Insert into MySQL):

```
String sql = "Insert INTO users (name, email)
```

```
values (?, ?)";
```

```
PreparedStatement pstmt = conn.prepareStatement(sql)
```

```
pstmt.setString(1, "Asha");
```

```
pstmt.setString(2, "asha@example.com");
```

```
pstmt.executeUpdate();
```

(23)

Q.23 : What is a ResultSet in JDBC and how is it used to retrieve data from a MySQL database? Briefly explain the use of next(), getString() and getInt() methods with an example.

Solution: ResultSet is an object in JDBC used to store and retrieve data returned from executing a SELECT query.

next() → Moves to the next row.

getString("column") → Gets a string value from the column.

getInt("column") → Gets an integer value from the column.

Example:

```
String query = "SELECT name, age FROM users";
```

```
ResultSet rs = stmt.executeQuery(query);
```

```
while (rs.next()) {
```

```
    String name = rs.getString("name");
```

```
    int age = rs.getInt("age");
```

```
    System.out.println(name + " - " + age);
```

Q.24 How does JPA manage the mapping between Java objects and relational tables?

Explain with an example using @Entity, @Id and @GeneratedValue annotations. Discuss the advantage of using JPA over raw JDBC.

Solution:

JPA is a specification that simplifies the interaction between Java objects and relational databases by mapping Java classes to database tables.

How JPA Maps Objects to Tables:

- @Entity : Marks a class as a JPA entity
- @Id : Marks the primary key of the entity
- @GeneratedValue : Automatically generates the value for the primary key.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Example :

```
import javax.persistence.* ;

@Entity
public class student {

    @Id
    @GeneratedValue (strategy = GenerationType.IDENTITY)
    private long id;
    private String name;
    private int age;
}
```

This maps the student class to a table named student with columns id, name and age.

Advantages of JPA over raw JDBC:

JPA Features	JDBC Limitation
1. Object-oriented mapping	1. Manual SQL and mapping
2. Automatic SQL generation	2. Developer writes all SQL manually

3. Transaction are easier

4. Entity relationships

5. Less boilerplate code

3. Manual transaction management

4. Must be managed manually in JDBC

5. Verbose code for queries and updates

(25)

Solution:

Simple CRUD App with Spring Boot and MySQL
(Student Management) →

1. Setup:

Entity class (student.java):

@Entity

public class student {

@Id

@GeneratedValue (strategy = GenerationType.IDENTITY)

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

private Long id;

private String name;

private int age;

Repository Interface :

public interface StudentRepository extends JpaRepository<

Student, Long> { }

Create (Add student) :

@PostMapping ("/students")

public Student addStudent (@RequestBody Student student) {

return repository.save(student); }

Read :

@GetMapping ("/students")

public List<Student> getAll () {

return repository.findAll();

}

```
@GetMapping ("/students/{id}")
public Student getById(@PathVariable Long id) {
    return repository.findById(id).orElse(null);
}
```

Update :

```
@PutMapping ("/students/{id}")
public Student update (@PathVariable Long id,
    @RequestBody Student s) {
    Student existing = repository.findById
    if (existing != null) {
        existing.setName(s.getName());
        existing.setAge(s.getAge());
        return repository.save(existing);
    }
    return null;
}
```

Delete :

```
@DeleteMapping ("/students/{id}")  
public void delete(@PathVariable Long id){  
    repository.deleteById(id);  
}
```

(27)

Solution:

Spring Boot and RESTful Services:

Spring Boot simplifies REST Development by:

- Auto-configuring everything
- No need for XML configuration
- Easy annotations like @RestController, @GetMapping

@PostMapping

- Built-in JSON handling with Jackson.

Basic REST controller Example:

@RestController

@RequestMapping("/students")

public class studentController {

 @GetMapping

 public List < Student > getStudents() {

 return studentService.getAllStudents();

 }

 @PostMapping

 public Student addStudent(@RequestBody Student student) {

 return studentService.saveStudent(student);

 }

}

(25)

Difference between @RestController and @Controller :

@Controller → Returns views (HTML/JSP)
→ Used in MVC web apps.

@RestController → Returns data (JSON/XML)
→ Used in REST APIs.

Restful API Example for Library System (Books)

HTTP Method	EndPoint	Action
GET	/books	Retrieve all books
GET	/books/{id}	Retrieve a specific book
POST	/books	Add a new book
PUT	/books/{id}	Update a book
DELETE	/books/{id}	Delete a book.

TOPIC NAME : _____

Comparison

Maven in Spring Boot Projects : -

Maven's Role :

Maven manages:

- Dependencies (via pom.xml)
- Build lifecycle (compile → test → package → install → deploy)

Structure of a Typical pom.xml :

<project>

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>student-app</artifactId>

<version>1.0.0</version>

<parent>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-parent</artifactId>

<version>3.0.0</version>

</parent>

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<dependency>

<groupId>mysql</groupId>

<artifactId>mysql-connector-java</artifactId>

</dependency>

</dependencies>

<build>

<plugins>

<plugin>

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE / /

```
<groupId> org.springframework.boot </groupId>  
<artifactId> spring-boot-maven-plugin </artifactId>
```

```
</plugin>
```

```
</plugin>
```

```
</build>
```

```
</project>
```

Spring Boot Starter Dependencies:

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- Simplifies setup by auto-configuring common dependencies.

Solution:

Feature	Maven	Gradle
Syntax	XML (pom.xml)	Groovy / Kotlin DSL (build.gradle)
Readability	Verbose and structured	Concise and flexible
Performance	Slower due to downloading and no caching	Faster
Dependency mgmt	Uses <dependency> blocks	Uses implementation, compileOnly etc
Build Scripts	Harder to customize	Easier with scriptable logic.
Learning Curve	Easier for beginners	Slightly steeper

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : ____/____/____

Example : build gradle for a simple Spring Boots
REST API .

```
plugin {
```

```
    id 'org.springframework.boot' version '3.2.0' .
```

```
    id 'io.spring.dependency-management' version '1.1.3'
```

```
    id 'java'
```

```
}
```

```
group = 'com.example'
```

```
version = '10.0.1-SNAPSHOT'
```

```
sourceCompatibility = '17'
```

```
repositories {
```

```
    mavenCentral()
```

```
}
```

```
dependencies {
```

```
    implementation 'org.springframework.boot:spring-boot-  
starter-web'
```

```
    testImplementation 'org.springframework.boot:spring-  
boot-starter-test'
```

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

```
-test {  
  Use JUnit Platform ( )  
}
```

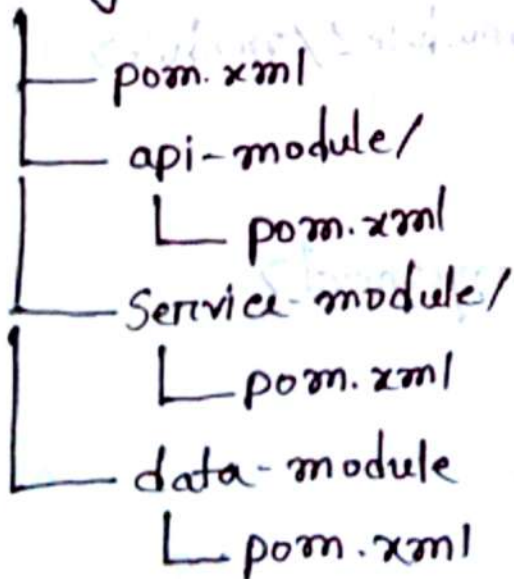
31

Solution:

A multi-module project in Spring Boot helps separate concerns, promote modularity and simplify testing and maintenance.

Typical Module Structure:

Springboot-multimodule-parent/



TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : ____/____/____

1. Parent Module - pom.xml (maven)

<project>

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>springboot-multimodule-parent</artifactId>

<packaging>pom</packaging>

<version>1.0.0</version>

<modules>

<module>data-module</module>

<module>service-module</module>

<module>api-module</module>

</modules>

<dependencyManagement>

<dependencies>

<dependency>

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

```
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-dependencies </artifactId>
<version> 3.2.0 </version>
```

```
<type> pom </type>
```

```
<scope> import </scope>
```

```
</dependency>
```

```
</dependencies>
```

```
</dependencyManagement>
```

```
<build>
```

```
<pluginManagement>
```

```
<plugins>
```

```
<plugin>
```

```
<groupId> org.springframework.boot </groupId>
```

```
<artifactId> spring-boot-maven-plugin </artifactId>
```

```
</plugin>
```

```
</plugins>
```

```
</pluginManagement>
```

```
</build>
```

```
</project>
```

2. Module: data-module/pom.xml

<project>

<parent>

<groupId>com.example</groupId>

<artifactId>springboot-multimodule-parent</artifactId>

<version>1.0.0</version>

</parent>

<artifactId>data-module</artifactId>

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

<groupId>mysql</groupId>

<artifactId>mysql-connector-java</artifactId>

</dependency>

</dependencies>

</project>

3. Module : service - module / pom.xml

```
<project>
  <parent>
    <groupId>com.example </groupId>
    <artifactId>springboot-multimodule-parent </artifactId>
    <version>1.0.0 </version>
  </parent>
  <artifactId>service-module </artifactId>
  <dependencies>
    <dependency>
      <groupId>com.example </groupId>
      <artifactId>data-module </artifactId>
    </dependency>
  </dependencies>
</project>
```

1. Module: api-module /pom.xml

<project>

<parent>

<groupId>com.example </groupId>

<artifactId>springboot-multimodule-parent </artifactId>

<version>1.0.0 </version>

</parent>

<artifactId>api-module </artifactId>

<dependencies>

<dependency>

<groupId>org.springframework.boot </groupId>

<artifactId>spring-boot-starter-web </artifactId>

</dependency>

</dependencies>

</project>

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Managing Dependencies Effectively:

- Use dependencyManagement in the parent pom.xml to centralize version control
- Only declare versions in parent, not in modules.
- Import modules in the order of dependency.